

Föreläsning 260505 – OOP (Objektorienterad Programmering)

1. Under -60 talet, program = algoritm + **data**
2. Under -80 talet, program = Modell av verkligheten
3. Om man betraktar **data** som objekt => OOP

Varför utvecklade man Objektorienterad programmering?

- Under -70 talet blev programmen alltför komplexa och svåra att underhålla, förnya och vidareutveckla.
- Redan i designstadiet behövde man ett annorlunda upplägg än tidigare. Fokuset ändrades från problemlösning till **modellering av verkligheten** – Objektorienterad design (UML).

Objekt, klass och metod

För att kunna avbilda den reella världen och för att kunna simulera verkligheten, består världen av **objekt** (människor, byggnader, bilar, böcker, skolor, anställda, kunder, varor, fakturor, bokningar, kurser osv.)

Objekt = dess egenskaper (attribut).

Objektet bil har attribut som fabrikat, modell, färg, årsmodell osv.

Attributen svarar på frågan: vad är det för bil.

Vi vill också veta vad man kan göra med en bil:

Genom att skapa **metoder**: att köra fram, att backa osv.

Klass: Summan av dess attribut och metoder.

Man skriver klassen en gång medan objekt (instans) kan skapas av klassen i obegränsat antal.

Klassen är som en sorts recept (t.ex. bil, pepparkaksform). Pepparkakorna eller bilmodellerna är objekten.

Titta på dessa länkar om Ni känner er osäkra på grunderna:

Hur klasser definieras och nya instanser skapas, hur man definierar egenskaper och hur man både anger värden och hämtar värden för en viss instans av klassen.

<https://learn.microsoft.com/sv-se/shows/c-fundamentals-for-absolute-beginners/15>

Vad händer exakt när du skapar en ny instans av en klass? Vad är en referens till en instans av en klass? Hur påverkar överföring av referensen till en metod en klass? Här granskas även överlagrade metoder, statiska metoder jämfört med instansmetoder och konstruktörer.

<https://learn.microsoft.com/sv-se/shows/c-fundamentals-for-absolute-beginners/16>

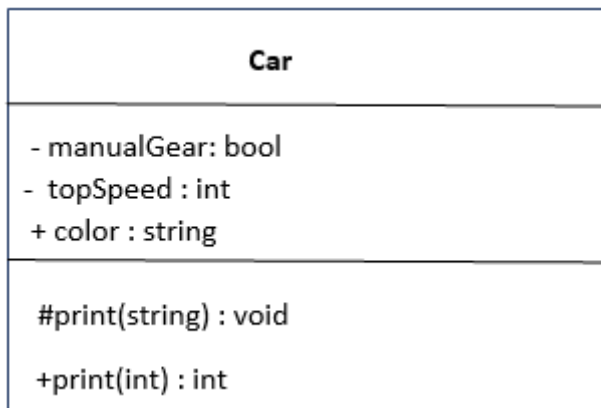
Fortsätter att lära ut begrepp om klasser (särskilt i det här fallet arv) genom att visa hur man använder arv i dina egna anpassade klasser. Lär dig mer om att åsidosätta virtuella funktioner, abstrakta basklasser och förseglade klasser.

<https://learn.microsoft.com/sv-se/shows/c-fundamentals-for-absolute-beginners/17>

Vid lite större eller mer komplexa system/program

- Använder man Klassdiagram (UML – Unified Modeling Language)

Se följande klass. Beskriv vilka variabler den har, vilka metoder den har, in-parametrar och returtyper från metoderna. Om variabler och metoder är private, public eller protected.



Variabler:

manualGear	private, typ bool
topSpeed	private, typ int
color	public, typ string

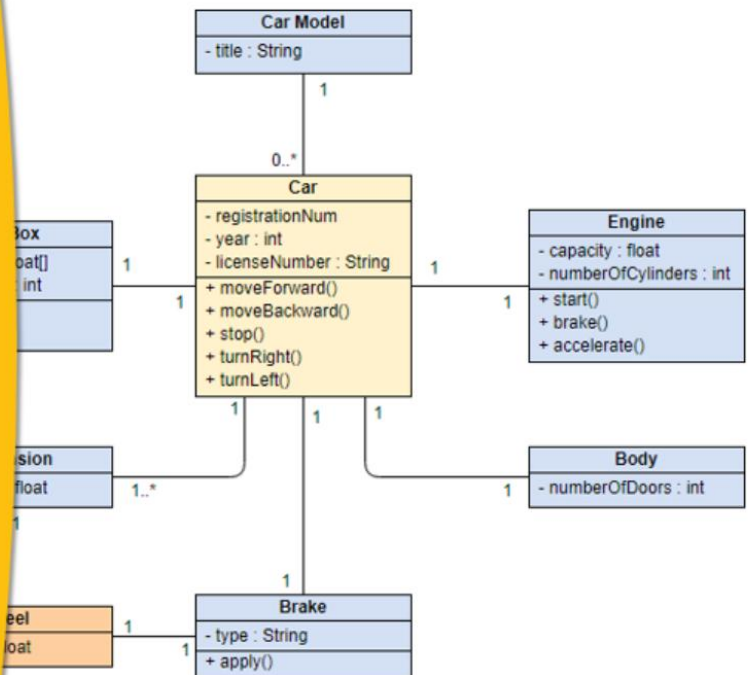
Metoder:

print(string)	protected, string in-parameter, returtyp void
print(int)	public, int in-parameter, returtyp int

Symbolerna -, + och # styr åtkomsten till medlemmarna (**Access modifiers**):

What is Class Diagram?

Visual Paradigm



Konstruktor:

Är en **metod** som körs när man skapar en **instans** av en klass.

Det är användbart eftersom nästan alla klasser innehåller **variabler som behöver initieras** när man skapar en instans.

Konstruktor definieras utan returtyp.

```
static void Main(string[] args)
{
    // Declare a new instance of the class Car, called the object: myNewCar
    Car myNewCar = new Car();

    Console.WriteLine(myNewCar.Make);    // Writes Nissan
}
```

```
class Car
{
    // Properties, egentligen metoder
    public string Make {get; set; }
    public string Model {get; set; }
    public int Year {get; set; }
    public string Color {get; set; }
    public double OriginalPrice {get; set; }

    // Konstruktor
    public Car()
    {
        this.Make = "Nissan";
    }

    public void MaxVelocity()
    {
    }

    public void Accelerate()
    {
    }

    public void DeAccelerate()
    {
    }
}
```

Konstruktorn är en typlös (även utan void) **medlemsfunktion med samma namn som klassen**. Klassen kan dock innehålla flera konstruktorer, men med olika antal parametrar, d v s de är då **överlagrade**.

Arv – Koncept

Basklass

Subklass

Arv – Relation mellan klasser

Arv – Kod - Subklassen anger dess basklass med ett kolon.

Arv – Konstruktör

```
Animal animal = new Animal(10);  
Console.WriteLine(animal.Weight); // 10  
  
Cat cat = new Cat(5);  
Console.WriteLine(cat.Weight); // 5
```

Arv – Single Inheritance

Alla klasser ärver direkt eller indirekt från basklassen Object, som bl.a. erbjuder metoden ToString.

- Ofta skriver man över ToString() i egna klasser för att få mer meningsfull utskrift.

```
class Person
{
    public string Name { get; set; } = "";
    public int Age { get; set; }

    public override string ToString()
    {
        return $"Person: {Name}, {Age} år";
    }
}
```

```
Person p = new Person { Name = "Anna", Age = 25 };
Console.WriteLine(p);
```

Får vi denna utskrift:

```
Anna, 25
```

Så p ovan anropar automatiskt p.ToString()

Övning

Bygg om Övn. 2 i klassens repo:

- med klasser
- dela upp huvudprogrammet i main med lämpliga metoder
- Ni kan själva lägga till egen funktionalitet om Ni vill
- Det är frivilligt att lämna in ert repo till oss lärare.

Lycka till!